

PROJECT RECORD: INCREMENTAL PAGERANK

Haopeng Zhang

Using dataset: [Wiki-Vote from Stanford](#)

A python implementation of incremental PageRank Algorithm

Theoretical Understanding of PageRank

Background

1998 年，斯坦福大学的博士生 Larry Page 和 Sergey Brin 创立了 Google 公司，其核心技术是通过 PageRank 技术对海量数据进行分析，利用网页相互连接的关系对网页进行组织，确定出每个网页的重要级别（PageRank）。当用户检索时，Google 找到符合要求的网页并按照他们的重要级别排序并以此向用户列出，这使得用户可以在前几条检索结果就找到需要的结果。对重要级别的直观理解是：如果网页 A 中存在一条链接指向网页 B，就认为网页 A 给网页 B 投了一票。因此，被链接的网页的重要级别与链接它的网页的重要级别相关，换句话说，如果网页 A 的重要级别是高的，那么网页 B 的重要级别也相应的是高的。

PageRank 成为了高效搜索的主要手段。在互联网的广泛使用下，PageRank 保证人们在生产、生活中获取信息的效率，并且在网络信息量日渐增长的时代中发挥着不可或缺的作用。

Mathematical Modeling

对于有 n 个网页的网络，定义 $n \times n$ 的邻接矩阵 $G = (g_{ij}) \in \mathbb{R}^{n \times n}$ ，若网页 j 有一个链接到网页 i ，则 $g_{ij} = 1$ ，否则 $g_{ij} = 0$ 。 $c_j = \sum_{i=1}^n g_{ij}$ 称为网页 j 的出度，即从网页 j 出发的链接的总数，类似的 $r_i = \sum_{j=1}^n g_{ij}$ 称为网页 i 的入度，即指向网页 i 的链接总数。

假设一个随机的网上冲浪过程，即每次看完当前网页后，有两种选择：

1. 在当前网页的链接中随机挑选一个，假设进行这一动作的概率是 p 。
2. 随机新开一个网页。

这在数学上是一个马尔可夫过程，且这样得随机冲浪过程一直进行下去，某一个网页被访问到的概率就是它的 PageRank。那么如果当前网页是 j ，下一次访问到网页 i 的概率的计算方式就是：

1. 若网页 j 存在一个链接指向网页 i ，
 1. 通过网页 j 上的链接访问网页 i 的概率就是 $p \times \frac{1}{c_j}$ 。
 2. 通过随机新开一个网页，打开的恰好是网页 i 的概率就是 $(1-p) \times \frac{1}{n}$ 。
 3. 访问到网页 i 的概率是 $p \times \frac{1}{c_j} + (1-p) \times \frac{1}{n}$ 。
2. 若网页 i 不在网页 j 的连接上，通过随机新开一个网页打开网页 i 的概率就是 $(1-p) \times \frac{1}{n}$ 。

由于 g_{ij} 为 1 或 0，表示网页 j 上是否存在一个链接指向网页 i ，下一次访问到网页 i 的概率可以表示为：

$$a_{ij} = g_{ij} \times [p \times \frac{1}{c_j} + (1-p) \times \frac{1}{n}] + (1 - g_{ij}) \times [(1-p) \times \frac{1}{n}] = \frac{pg_{ij}}{c_j} + \frac{1-p}{n} \quad (1)$$

任意两个网页间的转移概率构成一个转移矩阵 $A = (a_{ij})$ 。设 n 阶对角矩阵 $D = (d_{ij})$ 为各个网页出度的倒数（若 $c_j = 0$ ，意味着 $g_{ij} = 0$ ， d_{jj} 可设为 1）， $\vec{e} \in \mathbb{R}^n$ 为元素全为 1 的 n 维列向量，向量 $\vec{f}$ 满足：

$$f_j = \begin{cases} \frac{1-p}{n}, & c_j \neq 0 \\ \frac{1}{n}, & c_j = 0 \end{cases} \quad (2)$$

在这里， $\vec{f}$ 并不是一个完全相同的常数向量，而是为了解决悬挂节点而设计的。悬挂节点指的是没有出链的网页（即 $c_j = 0$ ）。如果直接按比例计算，这部分网页会导致系统中的概率总和不断流失。因此，当遇到悬挂节点（ $c_j = 0$ ）时，我们将其下一步访问任何网页的概率强行设为 $\frac{1}{n}$ ，相当于强制进行一次随机跳转，以此补偿跳转概率，确保整个网络的转移概率守恒。

那么转移矩阵 A 可以表示为以下形式：

$$A = pGD + \vec{e}\vec{f}^T \quad (3)$$

设 $\vec{x}^{(k)}$ 是 k 时刻访问各个网页的概率分布 ($\sum_i x_i^{(k)} = 1$)，那么下一时刻浏览各个网页的概率分布就是 $\vec{x}^{(k+1)} = A\vec{x}^{(k)}$ 。当这个过程无限进行下去，达到极限情况，即网页访问概率收敛到一个极限值，这个极限向量 $\vec{x}$ 就是各个网页的 PageRank，它满足 $A\vec{x} = \vec{x}$ ，且 $\sum_i x_i = 1$ 。

Power Method

幂法是求解大规模稀疏矩阵的主特征向量的可靠、唯一的选择。

Theoretical Bound of Network Perturbation

PageRank 向量 $\vec{x}$ 满足 $\sum_i^n x_i = 1$ (即 $\vec{e}^T \vec{x} = 1$)，且 $A\vec{x} = \vec{x}$ 。

对于包含悬挂节点（即出度 $c_j = 0$ 的节点）的网络， GD 必然存在全 0 列，因此直接将其视作列和为 1 的列随机矩阵是相互矛盾的。为了严谨地进行推导，我们需要引入悬挂节点指示向量 $\vec{d}$ ，当节点 j 为悬挂节点时 $d_j = 1$ ，否则 $d_j = 0$ 。

由于 $\vec{f}^T$ 的第 j 个分量可以写为 $f_j = \frac{1-p}{n} + \frac{p}{n}d_j$ ，转移矩阵 A 可重写为：

$$A = pGD + \vec{e} \left(\frac{1-p}{n} \vec{e}^T + \frac{p}{n} \vec{d}^T \right) = p \left(GD + \frac{1}{n} \vec{e} \vec{d}^T \right) + \frac{1-p}{n} \vec{e} \vec{e}^T \quad (4)$$

定义 $\bar{M} = GD + \frac{1}{n} \vec{e} \vec{d}^T$ ，由于 $\frac{1}{n} \vec{e} \vec{d}^T$ 正好补齐了悬挂节点缺失的概率，此时 $\bar{M}$ 是列和为 1 的列随机矩阵。

于是，PageRank 的特征方程可以改写为：

$$\vec{x} = A\vec{x} = p\bar{M}\vec{x} + \frac{1-p}{n} \vec{e} (\vec{e}^T \vec{x}) \quad (5)$$

因为 $\vec{e}^T \vec{x} = 1$ ，令 $\vec{v} = \frac{1}{n} \vec{e}$ （均匀分布向量），上式简化为：

$$\vec{x} = p\bar{M}\vec{x} + (1-p)\vec{v} \quad (6)$$

进一步将 $\vec{x}$ 表达为线性系统 $\vec{x} = (1-p)(I - p\bar{M})^{-1}\vec{v}$ 。而 $I - p\bar{M}$ 之所以可逆，是因为对于列随机矩阵 $\bar{M}$ 有 $|\lambda(\bar{M})| \leq \rho(\bar{M}) = 1$ ，进而 $I - p\bar{M}$ 的特征值均不为 0。记 $A_0 = I - p\bar{M}$ ，则 PageRank 向量 $\vec{x} = (1-p)A_0^{-1}\vec{v}$ 。这一形式将 $\vec{x}$ 表示为线性系统，并且 $\vec{v}$ 是均匀分布，网络结构的扰动仅导致 A_0 的变化，便于后续对 PageRank 向量受矩阵扰动影响的探究。

引入扰动 $\bar{M} \rightarrow \bar{M} + \Delta\bar{M}$ 导致 PageRank 向量变化： $\vec{x} \rightarrow \vec{x} + \Delta\vec{x}$ 。 A_0 相应变为 $A_0 - p\Delta\bar{M}$ ，有 $\vec{x} + \Delta\vec{x} = (1-p)(A_0 - p\Delta\bar{M})^{-1}\vec{v}$ 。对 $(A_0 - p\Delta\bar{M})^{-1}$ 进行近似一阶展开 $(A + E)^{-1} \approx A^{-1} - A^{-1}EA^{-1}$ 得到 $(A_0 - p\Delta\bar{M})^{-1} \approx A_0^{-1} + pA_0^{-1}\Delta\bar{M}A_0^{-1}$ 。进一步推导：

$$\begin{aligned} \Delta\vec{x} &= (1-p)(A_0^{-1} + pA_0^{-1}\Delta\bar{M}A_0^{-1} - A_0^{-1})\vec{v} \\ &= (1-p)pA_0^{-1}\Delta\bar{M}A_0^{-1}\vec{v} \\ &= pA_0^{-1}\Delta\bar{M}\vec{x} \end{aligned}$$

取 L_1 范数，有

$$\begin{aligned}
\|\Delta \vec{x}\|_1 &= \|pA_0^{-1}\Delta \bar{M}\vec{x}\|_1 \\
&\leq p\|A_0^{-1}\|_1\|\Delta \bar{M}\|_1\|\vec{x}\|_1 \\
&= p\|A_0^{-1}\|_1\|\Delta \bar{M}\|_1 \\
&= p\|\Delta \bar{M}\|_1\sum_{k=0}^{\infty}(p\bar{M})^k\|_1 \\
&\leq p\|\Delta \bar{M}\|_1\sum_{k=0}^{\infty}p^k\|\bar{M}^k\|_1 \\
&\leq p\|\Delta \bar{M}\|_1\sum_{k=0}^{\infty}p^k\|\bar{M}\|_1^k \\
&= p\|\Delta \bar{M}\|_1\sum_{k=0}^{\infty}p^k \\
&= \frac{p}{1-p}\|\Delta \bar{M}\|_1
\end{aligned}$$

综上，当列随机转移矩阵发生扰动 $\Delta \bar{M}$ 时，PageRank 向量的变化有如下界限：

$$\|\Delta \vec{x}\|_1 \leq \frac{p}{1-p}\|\Delta \bar{M}\|_1 \quad (7)$$

Environment Setup and Implementation of Static PageRank

用这些命令来配置环境：

```
conda create -n na_pr_env python=3.10
conda activate na_pr_env
pip install -r requirements.txt
```

静态 PageRank 算法的实现：

- `note.ipynb`
 - `load_data` 从指定文件（Wiki-Vote.txt 相同格式）加载数据。
 - `build_adjacency_matrix` 用 `edges`, `n_nodes`, `p` 来计算 A 。
 - `build_components` 用 `edges`, `n_nodes`, `p` 来计算 G （CSR 格式）、 D （一维数组）、 $\vec{f}$ 。
 - `power_method_A` 用 A , `n_nodes`, `max_iter`, `tol` 对 A 进行幂法得到 PageRank 向量。
 - `power_method_components` 用 G , D , e , f , `n_nodes`, `p`, `max_iter`, `tol` 不保存 A ，直接对 $pGD + \vec{e}\vec{f}^T$ 进行幂法来得到 PageRank 向量。
- `static_textbook_implementation_power_method.py`
 - 一个静态 PageRank 的幂法实现，在中小规模图上验证了其正确性。
 - 完全仿照教材中对 PageRank 的算法介绍，从原始数据开始一步步提取计算 G , D , $\vec{f}$ ，最终得到矩阵 A 。在此之后对 A 进行幂法，得到其主特征值，也就是各网页的 PageRank 向量。
 - 然而当面对大规模数据时，直接计算并全量保存矩阵 $A = pGD + \vec{e}\vec{f}^T$ 无疑是对时间、空间和算力的浪费，因此需要用 CSR 格式实现稀疏矩阵的存储和计算，并在更大规模的图上测试性能和正确性。
 - 在 Wiki-Vote 的 7115 个节点 + 103689 条有向边的网络拓扑下进行测试，全量保存矩阵 A 的计算过程所花费的时间要远远超过用 CSR 格式实现的稀疏矩阵的存储和计算，两种方法的具体表现和差异如下：

```
python src/scripts/static_textbook_implementation_power_method.py --data_file_path ./data/Wiki-Vote.txt
```

PAGERANK COMPUTATION REPORT

[*] Data Source : ./data/Wiki-Vote.txt
[*] Nodes : 7115
[*] Edges : 103689
[*] Parameters : p=0.85, tol=1e-06

[1] Running: Basic Implementation (Dense Matrix)...
Done! Time: 2.6537s

[2] Running: Component-wise Implementation (CSR Sparse)...
Done! Time: 0.0164s

METHOD: Basic (Dense)
Execution Time: 2.6537 seconds

Top 5 Ranking:

Rank	Node ID	Score
1	327	0.004607
2	410	0.003680
3	1333	0.003587
4	712	0.003284
5	906	0.002609

Vector Preview:[0.0002, 0.0008, 0.0018, 0.0022 ... 0.0001, 0.0001, 0.0001, 0.0001]

METHOD: Component-wise (CSR)
Execution Time: 0.0164 seconds

Top 5 Ranking:

Rank	Node ID	Score
1	327	0.004607
2	410	0.003680
3	1333	0.003587
4	712	0.003284
5	906	0.002609

Vector Preview:[0.0002, 0.0008, 0.0018, 0.0022 ... 0.0001, 0.0001, 0.0001, 0.0001]

END OF REPORT

Incremental PageRank Algorithm

Warm-Start PageRank

当网络结构发生扰动的时候，教材中提到了一种能够相较于随机化初始向量并重新执行幂法更快的方法是，使用扰动前得到的 PageRank 向量作为扰动后幂法的初始值。实验证明在 Wiki-Vote 数据集上，增添和删除边的比例在 0.1% 的情况下，Warm-Start 方法达到收敛所需要的迭代次数和迭代时间，较随机初始化向量进行幂法迭代均有下降，且迭代结果差异极小，可忽略不计。

```
python src/scripts/warm_start.py

=====
                        WARM-START INCREMENTAL PAGERANK REPORT
=====
[*] Loading original graph and calculatng PageRank vector...
[*] Perturbating (adding and dropping edges)...
-----

[1] Cold Start (Initializing with a random vector)...
    Done! Iterations: 27, Time: 0.004687s

[2] Warm-Start (Initialize with the previous PageRank vector)...
    Done! Iterations: 23, Time: 0.004432s

#####
METHOD: Cold Start
Execution Time: 0.004687 seconds
Iterations      : 27
#####

Top 5 Ranking:
-----
Rank      | Node ID | Score
-----
1         | 327     | 0.004569
2         | 712     | 0.003897
3         | 657     | 0.003718
4         | 410     | 0.003590
5         | 1333    | 0.003443
-----

Vector Preview:
[0.0002, 0.0008, 0.0017, 0.0021 ... 0.0000, 0.0000, 0.0000, 0.0000]

#####
METHOD: Warm-Start
Execution Time: 0.004432 seconds
Iterations      : 23
#####

Top 5 Ranking:
-----
Rank      | Node ID | Score
-----
1         | 327     | 0.004569
2         | 712     | 0.003897
3         | 657     | 0.003718
4         | 410     | 0.003590
```

```

5          | 1333          | 0.003443
-----

Vector Preview:
[0.0002, 0.0008, 0.0017, 0.0021 ... 0.0000, 0.0000, 0.0000, 0.0000]

=====
[3] Consistency & Performance Check:
    ||x_cold - x_warm||_1 = 2.6707e-09 (should be close to 0)
-----

Conclusion:
-> Iteration number reduced: 4 (14.81%)
-> Running time reduced    : 0.000256s (5.45%)

=====
                        END OF REPORT
=====

```

幂法求解主特征向量时，其所需的迭代次数直接取决于初始向量 $\vec{x}^0$ 和目标向量 $\vec{x}^*$ 之间的距离（通常用 L_1 范数衡量）初始差距越小，达到给定容差 ϵ 所需的迭代次数越少。根据矩阵扰动理论，当网络结构发生微小变化（如少量边的增删）时，新的转移矩阵对应的平稳分布向量 $\vec{x}_{new}$ 与旧向量 $\vec{x}_{old}$ 之间的差值界限极小。冷启动使用随机初始化的向量，需要经历完整的收敛过程，而 Warm-Start 方法直接将 $\vec{x}_{old}$ 作为新矩阵迭代的初始向量，由于 $\vec{x}_{old}$ 在数值上已经极其接近目标向量，Warm-Start 方法直接消除了绝大部分初始误差，使得算法仅需极少次数的迭代即可满足停止条件。

Local Push PageRank

一种增量式算法，基于残差推送理论求解线性系统的迭代方法。在代码实现中采用向量化批量推送以提高矩阵运算效率，其具体机制包含三个步骤：

- 初始状态：网络拓扑改变后，设新转移矩阵为 A_{new} 并将旧的 PageRank 向量赋给 $\vec{x}$ ，计算初始的全局残差向量 $\vec{r} = A_{new}\vec{x} - \vec{x}$ 。
- 迭代步骤（全量更新）：
 1. 将当前的残差向量整体无保留地累加到 PageRank 向量中： $\vec{x} \leftarrow \vec{x} + \vec{r}$ ；
 2. 将当前的残差分布通过新转移矩阵 A_{new} 向下传播，得到下一轮的残差向量： $\vec{r} \leftarrow A_{new}\vec{r}$ ；
- 停止条件：当残差向量的 L_1 范数 $\|\vec{r}\|_1$ 小于或等于规定容差 ϵ 时，算法停止，并对 $\vec{x}$ 进行 L_1 归一化以保证概率和为 1。

它能借助 python 中的 Numpy 等矩阵运算库获得极好的执行效率，而逐节点推送由于要在 Python 中频繁写循环和寻找最大值，执行速度会不增反减。

可以看到对比方法一（随机初始化并重新执行幂法），local push 算法在迭代次数和计算时间上均有显著提升，尽管相较于方法二（Warm-Start），local push 算法的提升较小，但仍然在保证 PageRank 结果一致的前提下，展现了效率上的进步。

```

python src/scripts/local_push.py

=====
                INCREMENTAL PAGERANK SCALABILITY TEST
=====
[*] 1/3 Calculating original converged PageRank vector...
[*] 2/3 Simulating network topology changes...
    -> Original Edges : 103689
    -> Added Edges    : 10 (0.01%)
    -> Dropped Edges  : 10 (0.01%)
    -> New Total Edges: 103689

[*] 3/3 Running PageRank Updates...

```

```
[1] Cold Restart (Scratch)
    -> ITERS: 026 | Time: 0.004029s

[2] Warm-Start (Old x as Init)
    -> ITERS: 018 | Time: 0.002950s

[3] Local Push
    -> ITERS: 017 | Time: 0.002605s
```

```
#####
METHOD: Cold Start
Execution Time: 0.004029 seconds
Iterations      : 26
#####
```

Top 5 Ranking:

Rank	Node ID	Score

1	327	0.004607
2	410	0.003680
3	1333	0.003584
4	712	0.003283
5	906	0.002608

Vector Preview:

[0.0002, 0.0008, 0.0018, 0.0021 ... 0.0001, 0.0001, 0.0001, 0.0001]

```
#####
METHOD: Warm-Start
Execution Time: 0.002950 seconds
Iterations      : 18
#####
```

Top 5 Ranking:

Rank	Node ID	Score

1	327	0.004607
2	410	0.003680
3	1333	0.003584
4	712	0.003283
5	906	0.002608

Vector Preview:

[0.0002, 0.0008, 0.0018, 0.0021 ... 0.0001, 0.0001, 0.0001, 0.0001]

```
#####
METHOD: Local Push
Execution Time: 0.002605 seconds
Iterations      : 17
#####
```

Top 5 Ranking:

```
-----
Rank      | Node ID    | Score
-----
1          | 327        | 0.004607
2          | 410        | 0.003680
3          | 1333       | 0.003584
4          | 712        | 0.003283
5          | 906        | 0.002608
-----

Vector Preview:
[0.0002, 0.0008, 0.0018, 0.0021 ... 0.0001, 0.0001, 0.0001, 0.0001]

=====
[*] Check ||Cold - Warm||_1: 8.0593e-10
[*] Check ||Cold - Local||_1 : 8.0593e-10
-----

[*] Warm-Start Speedup: 1.37x
[*] Local Push Speedup: 1.55x
=====

                        END OF TEST
=====
```

当图仅发生少量边变动时，初始残差 $\vec{r} = A_{new}\vec{x} - \vec{x}$ 的计算结果中，绝大多数节点的残差严格为零。非零残差仅出现在发生拓扑变动的节点及其直接邻居处。传统的全局幂法每次迭代都需要执行完整的矩阵向量乘法，其时间复杂度为 $O(|V| + |E|)$ 与全网规模绑定。而 Local Push 的每次单点更新操作复杂度仅为 $O(c_i)$ 。算法只在非零残差的局部子图中进行计算，直接跳过了未受影响的区域。在每次 Push 操作中，被传递的残差总和会乘以 p ，这意味着每执行一次推送，系统总残差的绝对值会严格损失 $1 - p$ 的比例。残差的快速收敛保证了活跃节点队列会迅速变空。Local Push 将计算复杂度从依赖全图规模转换为严格依赖实际发生变动的局部规模与给定的精度要求。在图变动比例较小的情况下，Local Push 方法能显著减少总体的浮点运算次数。

Experiments Design

Theoretical Bound Verification

实验验证在实际操作中，理论 upper 界 $\|\Delta\vec{x}\|_1 \leq \frac{p}{1-p} \|\Delta\vec{M}\|_1$ 是否成立，以及它到底有多紧密。

- 自变量：网络结构的扰动比例（同时或分别控制 `--add_ratio` 和 `--drop_ratio`，从极小值如 10^{-4} 逐渐增大到 10^{-1} ）。
- 因变量：记录实际排名误差 $\|\Delta\vec{x}\|_1$ ，并计算对应扰动下的理论 upper 界 $\frac{p}{1-p} \|\Delta\vec{M}\|_1$ 。
- 可视化：绘制横轴为扰动比例、纵轴为 L1 范数对数坐标的双折线图。

Performance & Scalability

目前的增量算法（Warm-Start 和 Residual Push）在不同的扰动规模下相对 Cold Start 的加速比可能会发生变化。

- 自变量：网络扰动比例（例如：0.01%, 0.05%, 0.1%, 0.5%, 1%, 5%, 10%）。
- 因变量：收敛所需的迭代次数 (Iterations)、运行时间 (Execution Time) 以及加速比 (Speedup)。
- 实验预期：在微小扰动下，Residual Push 和 Warm-Start 应该表现出巨大的速度优势。随着扰动比例增大，旧的向量作为初始状态/残差向量不再那么“精准”，增量算法的速度优势应该会逐渐衰减甚至退化。这能帮你探讨算法的适用范围。
- 编写一个批量测算的脚本 `run_performance_experiments.sh`，自动收集不同扰动比例下的运行数据并生成折线图。

Parameter Sensitivity

除了网络结构的变化，PageRank 本身的超参数对于迭代收敛速度也有显著影响。

- 阻尼系数 p : 如 0.70, 0.85, 0.90, 0.95。
 - 分析: p 越接近 1, 普通幂法的收敛速度越慢。此时, 增量算法能在多大程度上缓解因大 p 导致的严重性能开销?
- 收敛容忍度 ϵ : 如 10^{-6} , 10^{-9} , 10^{-12} 。
 - 分析: 在高精度要求下, Local Push 是否仍能保持稳定的加速比?

Ranking Stability

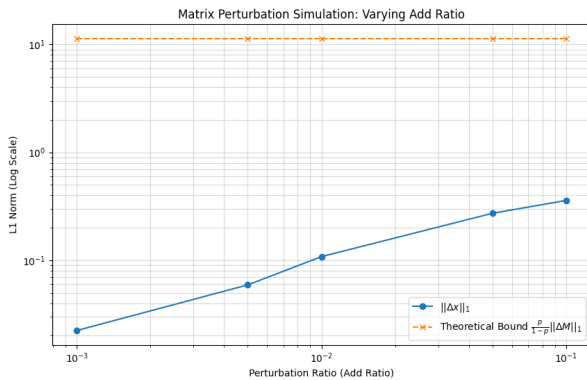
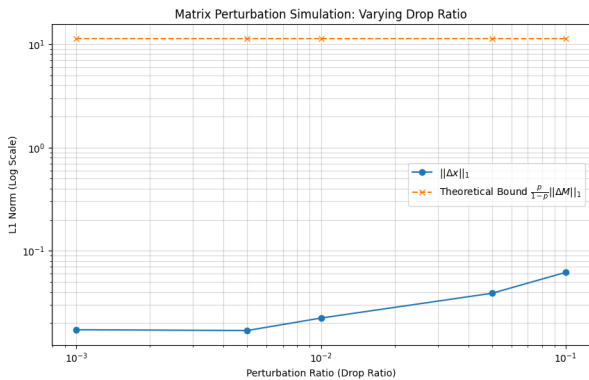
当前的理论主要使用 L_1 范数来衡量数值误差。但在真实的推荐系统或搜索引擎业务中, 用户更关心的是网络结构微调是否会导致头部排名 (Top-k) 发生剧烈洗牌。

- 因变量: 不同计算方法在新图上的排序一致性 (Kendall τ 相关系数、Spearman ρ)、Top-k 集合的 Jaccard 相似度与重叠率 (Overlap), 以及排名的位移 (Displacement)。
- 实验预期: 相比于 L_1 范数的敏感度, 实际的 Top-k 排序在小规模扰动下应当具备一定的“容错性”或“鲁棒性”。同时, 需要通过具体的业务指标验证 Warm-Start 和 Local Push 的计算结果不仅在数值上接近 Cold Start, 其实际排序结果也必须完全一致。

Results Analysis & Discussion

Perturbation

实验发现, 经典的一阶近似扰动界限 $\|\Delta \vec{x}\|_1 \leq \frac{p}{1-p} \|\Delta \vec{M}\|_1$ 在真实网络中表现出极强的保守性。下图中, 当我们以一定的比例增添边 (固定删除边的比例) 时, 可以观察到橙色折线, 也就是理论的扰动误差上界 $\frac{p}{1-p} \|\Delta \vec{M}\|_1$, 远高于蓝色的真实变化量折线。如果我们按一定比例删除边 (固定增添边的比例), 上述结果仍然成立。事实上, 当我们以一定比例增添和删除网络中的边时, 橙色线 (理论上界) 几乎平稳在 11.33 左右, 这背后有严密的数学必然性: 对于列随机矩阵, 其任意一列的绝对值变化总和 $\|\Delta \vec{M}\|_1$ 最大不可能超过 2, 当固定 $p = 0.85$ 的时候, 理论上界最大值刚好是 $\frac{0.85}{0.15} \times 2 \approx 11.33$ 。



Performance & Scalability

我固定 $p = 0.85$ 和 $\epsilon = 10^{-9}$, 测试了不同网络扰动比例下, 不同迭代方法到达收敛所需的迭代次数 (Iterations)、运行时间 (Execution Time) 以及 Warm-Start 和 Local Push 算法相对于重新执行幂法的加速比 (Speedup)。实验数据展示了不同 PageRank 计算策略在不同规模网络扰动下的性能表现及其适用边界。具体分析如下:

1. 低扰动区间内的增量计算优势

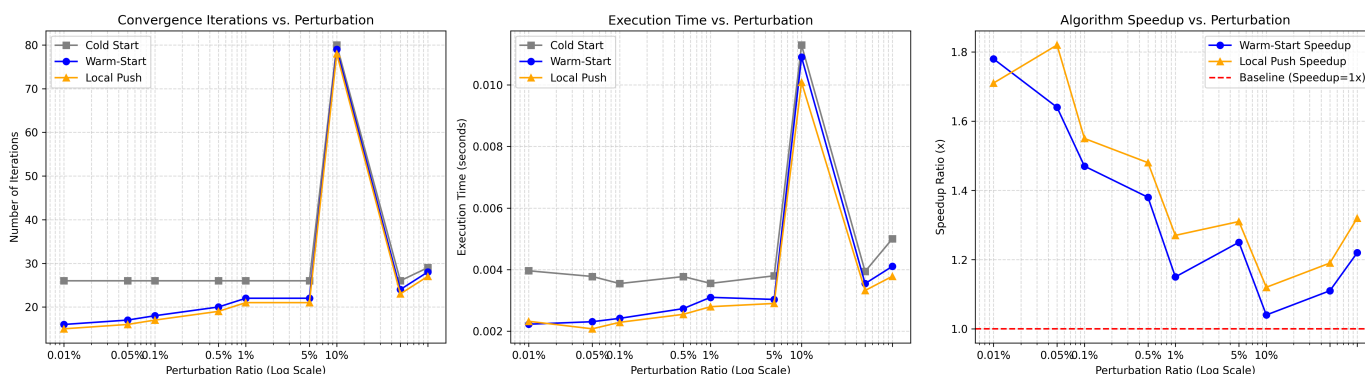
在网络拓扑变动比例较低 ($0.01\% \sim 0.1\%$) 时, Warm-Start 与 Local Push 所需的收敛迭代次数显著低于 Cold Start, 对应的绝对运行时间也大幅降低。这一结果验证了前期推导的矩阵扰动理论: 微小的转移矩阵扰动 (较小的 $\|\Delta \vec{M}\|_1$) 仅会产生极小的新旧 PageRank 向量偏差和极稀疏的初始残差。Warm-Start 和 Local Push 算法有效跳过了大量的全局冗余迭代。

2. 增量加速比的衰减规律与边界

图3直观地反映了增量算法的性能红利严格受制于网络变动规模。随着随机加删边比例的增大，系统的初始残差总量增加，旧状态与新平稳分布的距离拉大。因此，Warm-Start 和 Local Push 的加速比呈现出明显的阶梯式下降趋势。这说明增量式 PageRank 的适用场景是高频且微小的网络局部更新。当拓扑结构变动越过一定比例阈值时，增量式算法的时间收益会不断递减，最终趋近于全局重算。

3. 拓扑剧变下的矩阵谱特性影响与鲁棒性

在折线图的右侧（较高扰动比例处），三条曲线出现了一个同步的迭代激增峰值（逼近80次）。这可能是因为大规模的随机增删边操作破坏了原图的结构属性（例如改变了图的强连通分量分布，或缩小了转移矩阵第一与第二特征值之间的谱间隙），导致马尔可夫链的收敛难度全局性增加。但从数据可以看出，即便在这一极端不利的图结构下，Warm-Start 和 Local Push 的迭代次数与时间依然没有超过 Cold-Start，这证明了这两种增量实现方案在任何情况下都能保证不劣于全局重算的理论下限。



实验数据支撑了本项目的研究目标：对于真实演化的动态网络，利用向量化实现的增量式算法（特别是 Local Push 算法）能够显著加速 PageRank 的更新效率；但其核心前提是每次处理的拓扑扰动必须处于较低的比例范围内。

Parameter Sensitivity

为了进一步深化了对增量式 PageRank 算法性能边界的理解，我进行了超参数敏感性的相关实验。通过固定网络结构变化在 0.1% 并对阻尼系数 p 和收敛容忍度 ϵ 的交叉分析，可以得出以下结论：

1. 阻尼系数 p 对收敛性的主导作用：

实验结果验证了幂法收敛率由转移矩阵的谱间隙（Spectral Gap）决定的理论。随着 p 从 0.70 增加到 0.95，所有算法的迭代次数和运行时间均呈现显著上升趋势。这是因为 p 越接近 1，马尔可夫链的非主特征值对收敛的阻碍越强。

- 增量算法的鲁棒性：在 p 逐渐增大的过程中，Warm-Start 和 Local Push 始终保持在 Cold Start 曲线下方。
- 加速比的非线性波动：值得注意的是，加速比在 $p = 0.90$ 处达到峰值（Local Push 接近 1.45x），但在 $p = 0.95$ 时迅速回落。这说明在极高阻尼系数下，系统对拓扑扰动的敏感度极高，即使初始点距离真解很近，由于搜索空间的平坦性，微调所需的代价也开始逼近全局计算。

2. 容忍度 ϵ 对高精度更新的支撑：

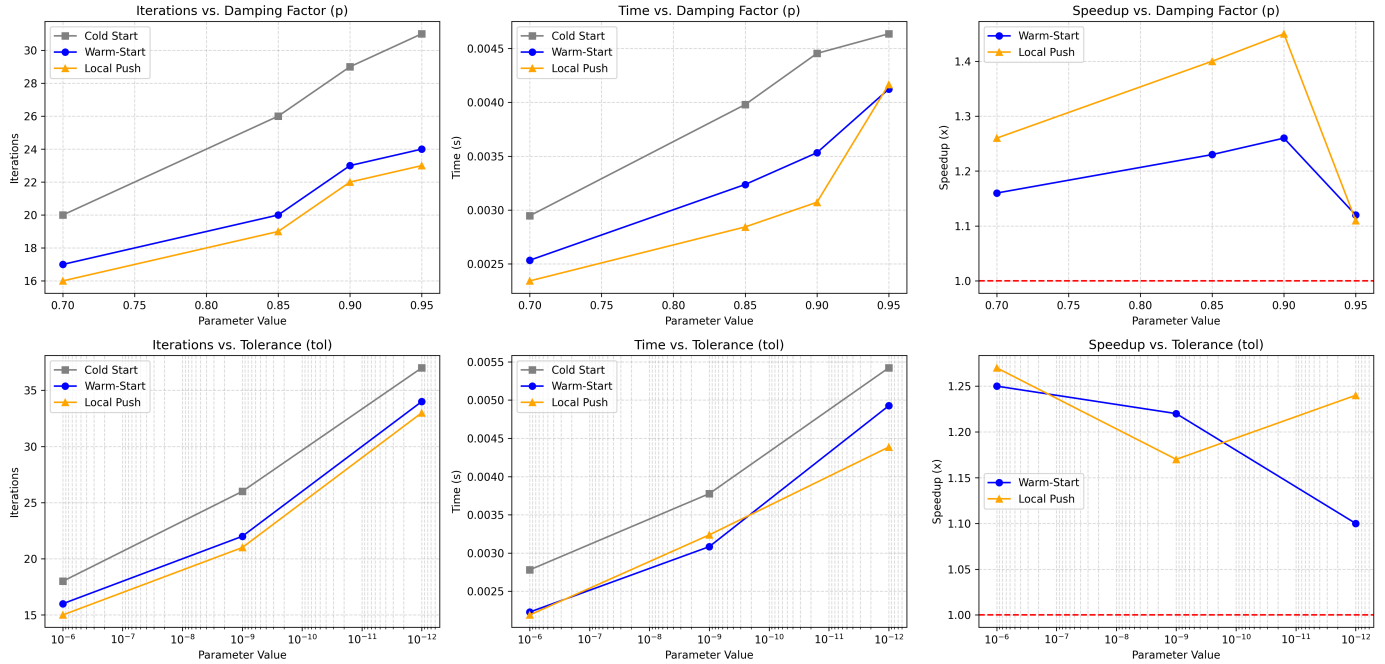
实验结果清晰地展示了计算精度与算力开销之间的对数线性关系。

- Local Push 的精度稳定性：在 10^{-6} 到 10^{-12} 的跨度内，Local Push 在迭代次数和运行时间上始终表现最优。
- 加速比的趋势分化：随着精度要求从 10^{-6} 提高到 10^{-12} ，Warm-Start 的加速比呈现下降趋势，而 Local Push 的加速比在经历中段波动后在 10^{-12} 处反弹，表现出更好的高精度适应性。这证明了基于残差传播的 Local Push 能够更精细地捕捉并消除由于拓扑扰动产生的微小概率偏差。

3. 算法工程特性的综合评价：

Local Push 的向量化残差推送算法在参数变化面前展现了极佳的工程灵活性。相比于简单的 Warm-Start，Local Push 不仅仅利用了旧的状态，更通过直接操作残差向量，使得它在处理高阻尼系数带来的慢收敛和高容忍度带来的长尾收敛问题时，拥有比传统幂法更高的搜索效率。

PageRank Parameter Sensitivity Analysis



实验数据表明，增量式算法（特别是 Local Push）在阻尼系数适中（约0.85-0.90）且精度要求较高（ 10^{-12} ）的动态网络更新场景中具有最强的竞争力。这契合实际工业界对 PageRank 实时性与准确性的双重需求。与此同时，在 p 极高或扰动过大的极端情况下，系统本身的数值不稳定性使增量算法的边际收益受到压缩。

Ranking Stability

数值误差（ L_1 范数）的增加是否直接等同于用户体验（搜索结果排序）的崩塌？我通过考察不同比例扰动下的排序稳定性 (Ranking Stability) 数据，得出了以下结论：

1. 增量算法的绝对排序安全性

数据表明，无论图结构的扰动比例是微小的 0.1% 还是剧烈的 10%，Warm-Start 和 Local Push 与 Cold Start 产生的 PageRank 向量之间的 Kendall τ 相关系数严格保持在 1.0（即完全正相关），且 Top-10、Top-50 和 Top-100 的节点排名位移 (Displacement) 均为 0。这从实际应用层面印证了增量算法能在提供加速比的同时，**确保 100% 的业务指标无损**。

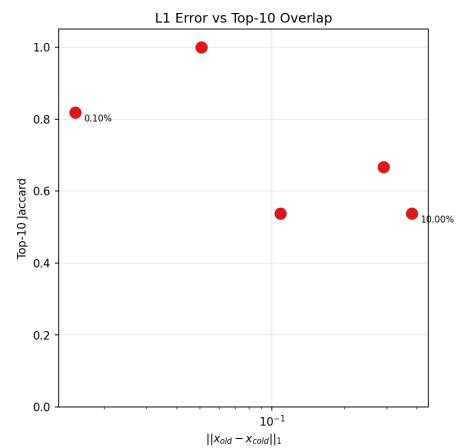
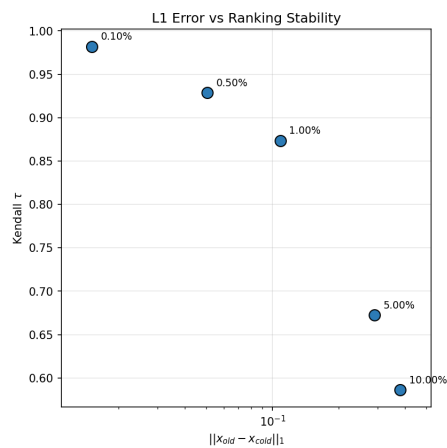
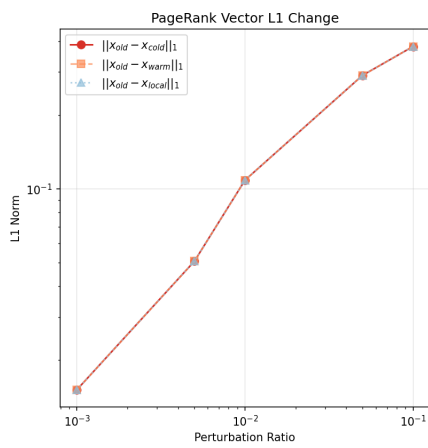
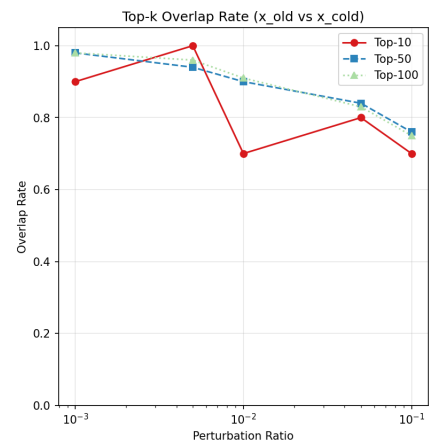
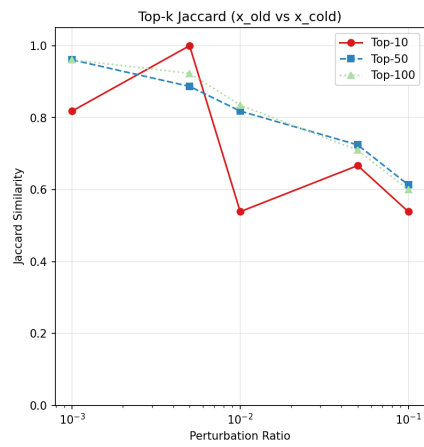
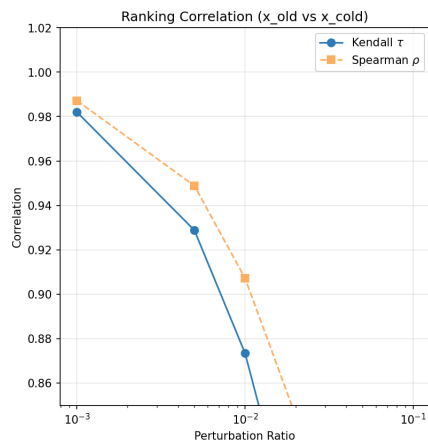
2. 拓扑微调下的排序鲁棒区域

在较低的扰动区间（0.1% ~ 0.5%），虽然新旧 PageRank 向量之间的 L_1 误差已经达到了 0.015 ~ 0.05，但整体排名的 Kendall τ 均保持在 0.92 以上，且 Top-10 节点的重叠率高达 90%-100%。即使头部节点发生位移，最大的名次变动也仅为 1。这意味着，在真实的演化网络中，少量的边增删引起的概率波动具有很强的“吸收性”，系统最具影响力的头部社群 (Top-k) 极其稳定。

3. 阈值效应与宏观排名洗牌

当扰动比例越过 1% 的阈值，甚至达到 5% 和 10% 时，排名的连贯性出现断崖式下跌。在 10% 的严重扰动下：

- Kendall τ 降至 0.58 左右，两张图的整体排序发生了剧烈分歧。
- Top-10 的节点重叠率仅剩 70%，这意味着搜索结果第一页被换掉了三分之一。
- Top-100 的节点平均排名位移达到了 13.8 名（最大位移高达 47 名）。此时，网络结构的剧烈变动不仅带来了 L_1 放大的数值漂移，更从根本上颠覆了网络的核心影响力分布。



综上所述, L_1 范数的数值误差并不能与最终排名的改变简单划等号。PageRank 的头部排名在网络微调时展现出了优秀的容错度, 但只要扰动比例打破了网络的内在核心结构, 排序洗牌将不可避免。